SED EXPLAINED
DATA STRUCTURES AND OPERATORS

Part—2

Continuing from last month's article on the subject, we now proceed to Sed data structures and operators.

Sed has a few powerful data structures and operators, which enable you to perform complex text-manipulation operations. Let's look at some of them.

Pattern space (p and P)

Pattern space is a memory area where *sed* copies a matched line or text first, before performing operations on it. Text-manipulation operations are (always) performed (only) on the contents of this pattern space. By default, the pattern space consists of a single line, copied as the line is read in. However, you can add multiple lines into the pattern space using line-reader operators. The '*p*' operator is used to print the entire pattern space. *P* is an operator like *p*, but it prints the content of the pattern space up to the first \n character that is found. To make this clear: let's have '*line1*\n*line2*' in the pattern space; then *p* prints both lines, but *P* prints only the first line.

Line readers n and N

The *n* operator is used to explicitly read a line. When used in a *sed* script, it reads a new line from the input, and replaces the current line in the pattern space. Let's try an example:

```
$ seq 10 | sed -n 'n;p'
2
4
6
8
10
```

When *sed* reads the first line, '*n*' explicitly replaces the contents of the pattern space with the second line, before it is printed with *p*. Similarly, when *sed* reads the third line, '*n*' explicitly reads in the fourth line, which is printed.

The second line-reader operator, *N*, appends the newly read line to the pattern space. Hence, the pattern space will contain *current_line*\n*read_line*. This is very useful to perform operations on multiple lines. For example:

```
$ seq 10 | sed -n 'N; s/\(.*\)\n\(.*\)/\2\n\1/; p'
2
1
4
3
6
5
8
7
10
9
```

Here, on each execution for a line, '*N*' is executed. Hence, in the first execution, the pattern space will be *1\n2* ; in the second execution, it will be *3\n4* and so on. The substitution operation is used to change the order of lines. The first and second lines are swapped with a \n in between. The first \(*.*\) matches the first line, while the second \(*.*\) after a \n matches the second line. In the replacement part, \2\n*1* is used to change the order.